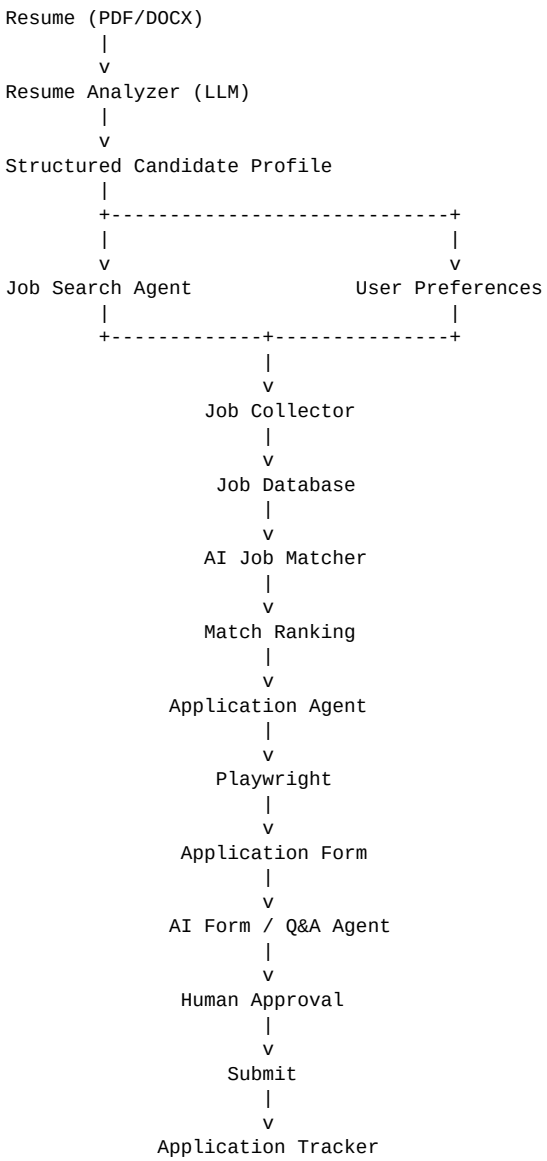
AI Job Application Agent

Production-Oriented Architecture, Workflow & Implementation Plan

Project objective: Build an AI-powered system that understands a candidate's resume and preferences, discovers relevant jobs across multiple sources, ranks them using semantic matching, prepares applications, assists with form filling and job-specific questions, and tracks application outcomes.

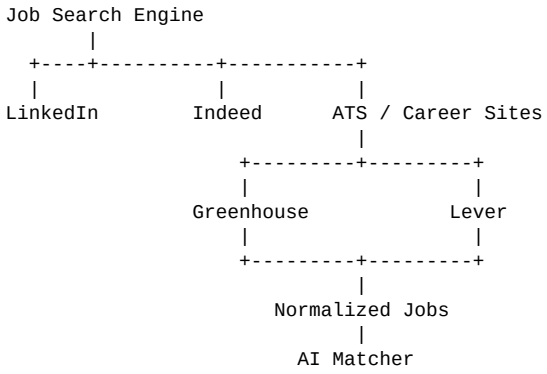
Recommended approach: Build progressively. Start with job discovery and matching, then add browser automation, AI-assisted application preparation, human approval, and finally carefully controlled auto-application.

1. System Overview



2. Core Design Principle

Do not make LinkedIn the central dependency. Treat each job source as a connector feeding a common job-normalization layer. This makes the system resilient when a particular site's UI, access rules, or automation behavior changes.



3. Recommended Technology Stack

Layer	Technology	Purpose
Backend	Python + FastAPI	REST APIs and orchestration
AI	LLM API	Resume analysis, matching, Q&A, tailoring
Agent orchestration	LangGraph	Stateful multi-step workflows
Browser automation	Playwright	Controlled browser interaction
Database	PostgreSQL	Users, jobs, matches, applications
Vector search	pgvector	Semantic resume/job retrieval
Parsing	PyMuPDF + python-docx	Extract resume content
Background jobs	Celery + Redis	Scheduled/asynchronous tasks
Frontend	React / Next.js	Dashboard and approval UI
Deployment	Docker	Reproducible deployment

4. Project Structure

```
ai-job-agent/
■■■ backend/
■   ■■■ app/
■   ■   ■■■ main.py
■   ■   ■■■ api/routes/
■   ■   ■   ■■■ resume.py
■   ■   ■   ■■■ jobs.py
■   ■   ■   ■■■ applications.py
■   ■   ■   ■■■ users.py
■   ■   ■■■ agents/
■   ■   ■   ■■■ job_search_agent.py
■   ■   ■   ■■■ job_match_agent.py
■   ■   ■   ■■■ application_agent.py
■   ■   ■   ■■■ question_agent.py
■   ■   ■■■ browser/
■   ■   ■   ■■■ browser.py
■   ■   ■   ■■■ linkedin.py
■   ■   ■   ■■■ indeed.py
■   ■   ■   ■■■ greenhouse.py
■   ■   ■■■ ai/
■   ■   ■   ■■■ resume_parser.py
■   ■   ■   ■■■ matcher.py
■   ■   ■   ■■■ question_answerer.py
■   ■   ■   ■■■ resume_tailor.py
■   ■   ■■■ models/
■   ■   ■   ■■■ user.py
■   ■   ■   ■■■ resume.py
■   ■   ■   ■■■ job.py
■   ■   ■   ■■■ application.py
■   ■   ■■■ services/
■   ■   ■   ■■■ job_service.py
■   ■   ■   ■■■ application_service.py
■   ■   ■   ■■■ resume_service.py
■   ■   ■■■ db/
■   ■   ■   ■■■ database.py
■   ■   ■   ■■■ migrations/
■   ■   ■■■ config.py
■   ■■■ requirements.txt
■   ■■■ Dockerfile
■■■ frontend/
■   ■■■ src/
■   ■   ■■■ components/
■   ■   ■■■ pages/
■   ■   ■■■ services/
■   ■   ■■■ App.jsx
■   ■■■ package.json
■■■ workers/
■   ■■■ scheduler.py
■   ■■■ tasks.py
■■■ tests/
■■■ docker-compose.yml
■■■ .env
■■■ README.md
■■■ .gitignore
```

5. Resume Analyzer

Upload a PDF or DOCX resume, extract its text, and use an LLM to convert it into a structured candidate profile. The analyzer should capture skills, experience, projects, education, certifications, target roles, and other job-search constraints.

```
{
  "candidate": {
    "name": "...",
    "education": [],
    "experience": [],
    "skills": [],
    "projects": [],
    "certifications": []
  },
  "target_roles": [
    "AI Engineer",
    "ML Engineer",
    "Backend Engineer"
```

```
  ]
}
```

6. Candidate Profile

```
{
  "target_roles": ["AI Engineer", "ML Engineer", "Backend Engineer"],
  "experience_level": "Entry Level",
  "locations": ["India", "Remote"],
  "skills": ["Python", "FastAPI", "LangChain", "LLM", "RAG", "Docker"],
  "remote_preference": true,
  "minimum_match_score": 75,
  "auto_apply": false
}
```

Keep **auto_apply=false** during development. Enable controlled submission only after the complete workflow is reliable and tested.

7. Job Search and Normalization

The search agent takes target roles, locations, experience level, skills, and other preferences and collects jobs from configured sources. Every source is converted into the same internal schema.

```
{
  "title": "AI Engineer",
  "company": "ABC Technologies",
  "location": "Bangalore, India",
  "remote": true,
  "experience": "0-2 years",
  "description": "...",
  "url": "...",
  "source": "greenhouse"
}
```

8. AI Job Matching

Use a hybrid matching system rather than relying on a single LLM judgment. Combine deterministic rules, embeddings/semantic similarity, and LLM reasoning.

Dimension	Suggested Weight
Skills Match	35%
Experience Match	20%
Role Match	20%
Location Match	10%
Education Match	5%
Responsibilities Match	10%

Example: AI Engineer — Match 91%. Skills 95%, Experience 85%, Role 100%, Location 100%, Education 80%, Responsibilities 90%. Explain both strengths and gaps, such as a preferred but missing Kubernetes requirement.

9. Job Ranking Pipeline

```
150 discovered
     ↓
120 valid
     ↓
94 unique
     ↓
65 relevant
     ↓
32 strong matches
     ↓
12 excellent matches
```

10. Application Agent

Use Playwright for controlled browser interaction. The agent should map detected form fields to known candidate-profile values instead of using brittle coordinate-based automation.

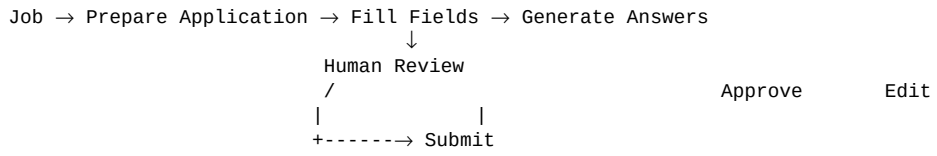
```
HTML field                          Candidate value
<input name="first_name">        → candidate.first_name
<input name="email">             → candidate.email
<input name="phone">             → candidate.phone
<input type="file">              → selected resume
```

11. AI Question Answering

For questions such as 'Why are you interested in this role?', provide the job description, company context, resume, and candidate profile to the LLM. Enforce grounding rules: never invent employment, education, skills, projects, achievements, or other experience. If the required information is unavailable, route the question to human review.

12. Human Approval

A strong production design uses a human-in-the-loop checkpoint before final submission, especially for unusual questions, work authorization, salary expectations, legal declarations, or anything that cannot be confidently answered from the candidate profile.



13. Application Tracking

```
applications
■■■ id
■■■ user_id
■■■ job_id
■■■ status
■■■ applied_at
■■■ resume_version
■■■ cover_letter
■■■ answers
■■■ application_url
■■■ source

Statuses:
SAVED → MATCHED → READY → APPROVED → SUBMITTED
                          ■
                           REJECTED / INTERVIEW / OFFER / WITHDRAWN
```

14. Database Schema

```
users
candidate_profiles
resumes
jobs
job_matches
applications

Key relationships:
users 1■■1 candidate_profiles
users 1■■N resumes
users 1■■N applications
jobs 1■■N job_matches
jobs 1■■N applications
```

15. FastAPI API Design

```
POST   /resume/upload
GET    /resume/profile

POST   /jobs/search
GET    /jobs
GET    /jobs/{id}
POST   /jobs/{id}/match

POST   /applications/{job_id}/prepare
GET    /applications
GET    /applications/{id}
POST   /applications/{id}/approve
POST   /applications/{id}/submit

PATCH  /profile
```

16. LangGraph Workflow

```
START
  ↓
Load Candidate Profile
  ↓
Search Jobs
  ↓
Normalize Jobs
  ↓
Deduplicate
  ↓
Calculate Match
  ↓
Score Above Threshold?
  ■■■ No → END
  ■■■ Yes
        ↓
Check Application
        ↓
Can Automate?
```

```
■■■ No → Human Review
■■■ Yes → Fill Form
                ↓
          Generate Answers
                ↓
            Human Review
                ↓
              Submit
                ↓
           Save Result
                ↓
               END
```

17. Scheduler

Once the core workflow is stable, add scheduled discovery. For example, a morning job search can collect and rank new jobs, while an evening workflow can process applications that the user has approved.

18. Security

Treat account access and personal data as sensitive application infrastructure. Do not store passwords in plain text. Prefer supported authentication methods where available, encrypt stored secrets/session information, keep API keys in environment variables or a secret manager, and never commit credentials to source control.

19. Development Roadmap

Version	Build
V1 — MVP	Resume upload → structured profile → job discovery → normalization → AI matching → dashboard
V2	Playwright application-page detection and profile-based autofill
V3	AI question answering, resume tailoring, cover-letter generation
V4	Human approval → submission → application tracking
V5	Scheduled discovery, notifications, and carefully controlled auto-apply

20. Recommended Build Order

1. Set up Python, FastAPI, PostgreSQL and the React frontend.

2. Implement resume upload and PDF/DOCX text extraction.

3. Create the structured candidate-profile schema.

4. Build job-source adapters and a normalized Job model.

5. Implement deduplication and basic deterministic filtering.

6. Add semantic/LLM job matching and an explainable match score.

7. Build the dashboard for discovered and ranked jobs.

8. Add Playwright and one application source at a time.

9. Implement robust form-field mapping and profile autofill.

10. Add grounded AI answers for application questions.

11. Add the human approval interface.

12. Implement submission and application-status tracking.

13. Add background workers and scheduled job discovery.

14. Test failure cases, authentication, duplicate applications, and site changes.

21. Final Product Concept

The finished system should behave like a personal job-search and application assistant: understand the candidate, continuously find relevant opportunities, explain why each opportunity matches, prepare applications using grounded candidate information, request human approval where judgment is required, submit approved applications through supported workflows, and maintain a complete application history.

Portfolio description: “Built an AI-powered job search and application system that parses candidate resumes, discovers and ranks jobs using semantic matching, prepares applications, fills application forms through browser automation, generates grounded responses to job-specific questions, and tracks application outcomes.”